

Módulo: Restrições e Regras de Integridade em Bancos de Dados Relacionais

Curso: Banco de Dados Relacionais aplicado à Administração Tributária **Público-alvo:** Profissionais da Secretaria de Estado da Fazenda da Paraíba (SEFAZ-PB)

SGBD utilizado: Microsoft SQL Server (T-SQL) — ferramenta: SQL Server

Management Studio (SSMS) **Carga horária sugerida:** 8 horas (4h teoria + 4h prática)

Sumário

1. [Objetivos do Módulo](#)
 2. [Por que Integridade de Dados Importa para a Administração Tributária](#)
 3. [Fundamentos Teóricos](#)
 4. [Restrições \(Constraints\) em T-SQL](#)
 5. [Tutorial: Preparando o Ambiente no SQL Server Management Studio](#)
 6. [Modelo de Dados de Referência do Curso](#)
 7. [Roteiro Prático 1 — Construindo o Esquema com Constraints](#)
 8. [Roteiro Prático 2 — Testando Violações de Integridade](#)
 9. [Roteiro Prático 3 — Regras de Negócio Fiscais com CHECK e Triggers](#)
 10. [Exercícios Práticos](#)
 11. [Estudos de Caso — Erros Reais de Integridade em Ambientes Fiscais](#)
 12. [Glossário](#)
 13. [Referências](#)
-

1. Objetivos do Módulo

Ao final deste módulo, o participante será capaz de:

- Explicar os quatro tipos clássicos de integridade de dados (domínio, entidade, referencial e definida pelo usuário).

- Utilizar o **SQL Server Management Studio (SSMS)** para criar bancos, executar scripts e inspecionar constraints visualmente.
 - Implementar restrições **NOT NULL**, **UNIQUE**, **PRIMARY KEY**, **FOREIGN KEY**, **CHECK** e **DEFAULT** em T-SQL.
 - Escolher corretamente as ações **ON DELETE / ON UPDATE** (**CASCADE**, **NO ACTION**, **SET NULL**) para relacionamentos entre documentos fiscais.
 - Modelar, com constraints apropriadas, um esquema simplificado que represente **NF-e**, **NFC-e**, **CT-e** e registros de **EFD** (Escrituração Fiscal Digital).
 - Escrever *triggers* em T-SQL para regras de negócio que exigem validação entre múltiplas linhas ou tabelas.
 - Diagnosticar e corrigir violações de integridade comuns em bases de dados fiscais.
-

2. Por que Integridade de Dados Importa para a Administração Tributária

Documentos fiscais eletrônicos (NF-e, NFC-e, CT-e) e escriturações (EFD ICMS/IPI, EFD Contribuições) alimentam bases de dados que sustentam:

- **Fiscalização e cruzamento de informações** (malha fiscal, cruzamento NF-e x EFD x GIA).
- **Cálculo e cobrança de tributos** (ICMS, substituição tributária, diferencial de alíquota).
- **Auditoria e combate à sonegação** (notas "frias", CT-e sem NF-e vinculada, cancelamentos fora de prazo).

Um erro de integridade — por exemplo, um item de NF-e referenciando um documento inexistente, ou um valor de imposto negativo — pode gerar:

- Inconsistências em relatórios gerenciais e no cruzamento de dados;
- Créditos ou débitos tributários incorretos;
- Falhas em auditorias e possíveis contestações judiciais por parte do contribuinte.

Em um ambiente tributário, a integridade de dados é essencial porque o sistema não armazena apenas informações: ele produz evidência para fiscalização, cálculo de tributos e análise de risco. Se um registro estiver inconsistente, a decisão do fiscal ou do auditor pode ser tomada com base em um dado incorreto.

Exemplo prático: tabelas fiscais simples

Considere um cenário em que uma NF-e possui vários itens e cada item precisa estar vinculado corretamente à nota fiscal correspondente:

```
CREATE TABLE contribuinte (  
    id_contribuinte BIGINT PRIMARY KEY,  
    cnpj_cpf        VARCHAR(14) NOT NULL UNIQUE,  
    razao_social     VARCHAR(150) NOT NULL  
);  
  
CREATE TABLE nfe (  
    id_nfe           BIGINT PRIMARY KEY,  
    chave_acesso     CHAR(44) NOT NULL UNIQUE,  
    id_contribuinte_emitente BIGINT NOT NULL,  
    valor_total      DECIMAL(12,2) NOT NULL,  
    CONSTRAINT fk_nfe_emitente FOREIGN KEY (id_contribuinte_emitente)  
        REFERENCES contribuinte(id_contribuinte)  
);  
  
CREATE TABLE item_nfe (  
    id_item          BIGINT PRIMARY KEY,  
    id_nfe           BIGINT NOT NULL,  
    valor_item       DECIMAL(12,2) NOT NULL,  
    CONSTRAINT fk_item_nfe_nfe FOREIGN KEY (id_nfe)  
        REFERENCES nfe(id_nfe)  
);  
  
CREATE TABLE cte (  
    id_cte           BIGINT PRIMARY KEY,  
    id_nfe_vinculada BIGINT NULL,  
    CONSTRAINT fk_cte_nfe FOREIGN KEY (id_nfe_vinculada)  
        REFERENCES nfe(id_nfe)  
);
```

Nesse modelo, a integridade referencial evita que um item de NF-e ou um CT-e aponte para uma nota que não existe. Isso é fundamental para preservar a coerência dos registros fiscais.

Consultas que ajudam a identificar problemas

Mesmo com as restrições em vigor, é importante analisar os dados para detectar inconsistências históricas ou falhas de importação.

1. Identificar itens de NF-e sem nota correspondente:

```
SELECT i.id_item, i.id_nfe
FROM item_nfe AS i
LEFT JOIN nfe AS n ON n.id_nfe = i.id_nfe
WHERE n.id_nfe IS NULL;
```

2. Encontrar notas fiscais cuja soma dos itens não bate com o valor total:

```
SELECT n.id_nfe,
       n.valor_total,
       SUM(i.valor_item) AS soma_itens
FROM nfe AS n
JOIN item_nfe AS i ON i.id_nfe = n.id_nfe
GROUP BY n.id_nfe, n.valor_total
HAVING SUM(i.valor_item) <> n.valor_total;
```

3. Localizar CT-e vinculados a uma NF-e inexistente:

```
SELECT c.id_cte, c.id_nfe_vinculada
FROM cte AS c
LEFT JOIN nfe AS n ON n.id_nfe = c.id_nfe_vinculada
WHERE c.id_nfe_vinculada IS NOT NULL
      AND n.id_nfe IS NULL;
```

Essas consultas demonstram que a integridade não serve apenas para impedir erros na inserção de dados, mas também para apoiar a auditoria, a fiscalização e a análise da qualidade das informações armazenadas.

Restrições de integridade não são apenas um requisito técnico — são um mecanismo de conformidade, qualidade de informação e defesa jurídica dos dados públicos.

3. Fundamentos Teóricos

3.1 Integridade de Domínio

Garante que cada coluna armazene apenas valores válidos dentro de um domínio (tipo, formato, faixa de valores).

Exemplo fiscal: o campo `uf` de um contribuinte deve pertencer ao conjunto das 27 UFs brasileiras; o campo `valor_total` de uma NF-e não pode ser negativo.

3.2 Integridade de Entidade

Garante que cada linha de uma tabela seja **unicamente identificável** — nenhuma chave primária pode ser nula ou duplicada.

Exemplo fiscal: a **chave de acesso** de uma NF-e (44 dígitos) identifica de forma única o documento em todo o território nacional; não pode haver duas linhas com a mesma chave de acesso.

3.3 Integridade Referencial

Garante que uma referência entre tabelas (chave estrangeira) sempre aponte para um registro existente na tabela referenciada.

Exemplo fiscal: todo `item_nfe` deve referenciar uma `nfe` existente; um `cte` que transporta mercadorias de uma NF-e deve referenciar uma NF-e efetivamente emitida.

3.4 Integridade Definida pelo Usuário (Regras de Negócio)

Regras específicas do domínio de aplicação, que vão além dos tipos e relacionamentos básicos, geralmente implementadas com `CHECK`, `TRIGGER` ou na camada de aplicação.

Exemplos fiscais:

- Uma NF-e cancelada não pode ser reaberta (transição de status irreversível).
- A soma dos valores dos itens deve ser igual ao valor total do documento.
- Uma NFC-e não pode ter CT-e vinculado (regra de segmento).

4. Restrições (Constraints) em T-SQL

Constraint	Tipo de Integridade	Função
NOT NULL	Domínio	Impede valor ausente em campo obrigatório
UNIQUE	Entidade	Impede valores duplicados em uma coluna (não-PK)
PRIMARY KEY	Entidade	Identificador único e não nulo da linha
FOREIGN KEY	Referencial	Garante que o valor exista na tabela referenciada
CHECK	Definida pelo usuário	Valida uma condição lógica sobre o valor da coluna/linha
DEFAULT	Domínio (apoio)	Preenche valor padrão quando não informado

4.1 Sintaxe Básica em T-SQL

No SQL Server, o autoincremento é feito com `IDENTITY(inicial, incremento)`, e não existe operador nativo de expressão regular no `CHECK` — usamos o operador `LIKE` com classes de caracteres (`[0-9]`, `[^0-9]`, etc.).

```
CREATE TABLE contribuinte (
    id_contribuinte    BIGINT IDENTITY(1,1) NOT NULL,
    cnpj_cpf           VARCHAR(14) NOT NULL,
    razao_social       VARCHAR(150) NOT NULL,
    uf                 CHAR(2) NOT NULL,
    situacao_cadastral VARCHAR(20) NOT NULL CONSTRAINT df_situacao DEFAULT
('ATIVO'),
    CONSTRAINT pk_contribuinte PRIMARY KEY (id_contribuinte),
    CONSTRAINT uq_contribuinte_doc UNIQUE (cnpj_cpf),
    CONSTRAINT ck_situacao CHECK (situacao_cadastral IN
('ATIVO', 'SUSPENSO', 'BAIXADO', 'INAPTO')),
    CONSTRAINT ck_uf CHECK (uf IN
('AC', 'AL', 'AM', 'AP', 'BA', 'CE', 'DF', 'ES', 'GO', 'MA', 'MG',
'MS', 'MT', 'PA', 'PB', 'PE', 'PI', 'PR', 'RJ', 'RN', 'RO', 'RR',
'RS', 'SC', 'SE', 'SP', 'TO'))
);
```

4.2 Chaves Estrangeiras e Ações Referenciais

```
CONSTRAINT fk_nfe_emitente
  FOREIGN KEY (id_contribuinte_emitente)
  REFERENCES contribuinte (id_contribuinte)
  ON DELETE NO ACTION
  ON UPDATE CASCADE
```

Ação (T-SQL)	Comportamento	Uso típico em contexto fiscal
NO ACTION	Impede a exclusão/alteração do "pai" se existir "filho" (é o padrão do SQL Server)	Impedir exclusão de um contribuinte que já emitiu documentos fiscais
CASCADE	Propaga a exclusão/alteração para os "filhos"	Excluir automaticamente os item_nfe ao excluir a nfe (em ambiente de testes)
SET NULL	Zera a referência no "filho"	Um documento pode ficar "sem transportadora" se a transportadora for removida do cadastro

Atenção — particularidade do SQL Server: o SQL Server **não permite múltiplos caminhos de **CASCADE/SET NULL** que levem à mesma tabela** (erro *"may cause cycles or multiple cascade paths"*). Nesses casos, é necessário usar **NO ACTION** e resolver a exclusão em duas etapas (ou via aplicação/**TRIGGER INSTEAD OF DELETE**). Isso é comum em nosso modelo, pois **nfe** é referenciada tanto por **item_nfe** quanto por **cte** e **efd_registro**.

Recomendação prática para bases fiscais de produção: preferir **NO ACTION** para preservar o histórico fiscal. **CASCADE** em exclusão é perigoso em bases de auditoria — documentos fiscais, uma vez emitidos, normalmente não devem ser fisicamente apagados (usa-se **cancelamento lógico** via coluna **situacao**, não **DELETE**).

5. Tutorial: Preparando o Ambiente no SQL Server Management Studio

Este tutorial assume o **SSMS** já instalado e conectado a uma instância de SQL Server (local, **LocalDB**, ou um servidor de homologação disponibilizado pela SEFAZ-PB).

5.1 Conectando ao servidor

1. Abra o **SQL Server Management Studio**.
2. Na janela **Connect to Server**:
 - *Server type*: Database Engine
 - *Server name*: informe o nome/instância (ex.: **localhost**, **.\SQLEXPRESS**, ou o endereço fornecido pela equipe de TI da SEFAZ-PB).
 - *Authentication*: **Windows Authentication** (mais comum em ambiente corporativo) ou **SQL Server Authentication** (usuário e senha).
3. Clique em **Connect**.

5.2 Criando o banco de dados do curso

1. No **Object Explorer** (painel à esquerda), clique com o botão direito em **Databases** → **New Database...**
2. Em *Database name*, digite **curso_integridade_fiscal**.
3. Clique em **OK**.

Alternativamente, pelo T-SQL (recomendado, pois documenta o passo em script):

```
CREATE DATABASE curso_integridade_fiscal;  
GO
```

5.3 Abrindo uma janela de consulta (New Query)

1. Clique com o botão direito sobre o banco **curso_integridade_fiscal** criado → **New Query**.
 - Isso garante que o contexto (**USE curso_integridade_fiscal;**) já esteja correto.
2. Verifique, no topo da janela de consulta, se o combo de banco de dados (ao lado do botão *Execute*) está mostrando **curso_integridade_fiscal**. Caso contrário, selecione manualmente ou execute:


```
USE curso_integridade_fiscal;  
GO
```

5.4 Executando scripts

- Para executar **todo o script** da janela: pressione **F5** ou clique em **Execute**.
- Para executar **apenas um trecho selecionado**: selecione o texto com o mouse e pressione **F5**.
- O separador **GO** divide o script em **lotes (batches)** — é obrigatório entre a criação de um objeto (ex.: **CREATE TRIGGER**) e o comando seguinte, pois **CREATE TRIGGER/CREATE FUNCTION/CREATE PROCEDURE** precisam ser o único comando do lote.

5.5 Lendo mensagens de erro

- O painel **Messages**, na parte inferior, mostra o resultado da execução e as mensagens de erro do SQL Server, no formato:

```
Msg 547, Level 16, State 0, Line 12  
The INSERT statement conflicted with the CHECK constraint "ck_nfe_valor_positivo".  
The conflict occurred in database "curso_integridade_fiscal", table "dbo.nfe",  
column 'valor_total'.
```

- O **número da mensagem** (**Msg 547**) e o **nome da constraint** citada na mensagem são a forma mais rápida de identificar qual regra foi violada — anote-os durante os exercícios do Roteiro Prático 2.

Código comum	Situação
Msg 2627	Violação de PRIMARY KEY ou UNIQUE (chave duplicada)
Msg 547	Violação de CHECK ou FOREIGN KEY
Msg 515	Tentativa de inserir NULL em coluna NOT NULL

5.6 Inspeccionando constraints pela interface gráfica (sem escrever SQL)

1. No **Object Explorer**, expanda `curso_integridade_fiscal` → **Tables** → a tabela desejada (ex.: `dbo.nfe`) → **Keys** e **Constraints**.
 - Em **Keys**, aparecem **PRIMARY KEY** e **FOREIGN KEY**.
 - Em **Constraints**, aparecem **CHECK** e **DEFAULT**.
2. Clique com o botão direito em qualquer constraint → **Script Key as** → **CREATE To** → **New Query Editor Window** para ver a definição exata gerada pelo SQL Server.
3. Para visualizar o **diagrama de relacionamentos**: clique com o botão direito em **Database Diagrams** → **New Database Diagram** e arraste as tabelas desejadas — o SSMS desenha automaticamente as linhas de **FOREIGN KEY**.

5.7 Consultando o catálogo de constraints via T-SQL

Alternativa em script (útil para auditoria e para os exercícios do módulo):

```
SELECT
    tc.TABLE_NAME,
    tc.CONSTRAINT_NAME,
    tc.CONSTRAINT_TYPE
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
ORDER BY tc.TABLE_NAME, tc.CONSTRAINT_TYPE;
```

6. Modelo de Dados de Referência do Curso

Um modelo simplificado, suficiente para os exercícios, cobrindo os quatro tipos de documento mencionados:

```
contribuinte (id_contribuinte PK)
|
|—< nfe (id_nfe PK, chave_acesso UQ, id_contribuinte_emitente FK,
id_contribuinte_destinatario FK)
|   |—< item_nfe (id_item PK, id_nfe FK, ncm, cfop, valor_item)
|   |—< nfce (id_nfce PK, chave_acesso UQ, id_contribuinte_emitente FK)
|       |—< item_nfce (id_item PK, id_nfce FK, ncm, cfop, valor_item)
|   |—< cte (id_cte PK, chave_acesso UQ, id_contribuinte_emitente FK,
```

```
id_nfe_vinculada FK)
```

```
    < efd_registro (id_registro PK, id_contribuinte FK, tipo_registro,  
    periodo_apuracao, id_nfe_referenciada FK)
```

Observações de modelagem:

- `chave_acesso` possui **restrição UNIQUE** e formato de 44 dígitos numéricos — combina integridade de domínio (**CHECK** de formato, via **LIKE**) com integridade de entidade (**UNIQUE**).
- `cte.id_nfe_vinculada` é opcional (nem todo CT-e carrega uma única NF-e), mas quando presente deve apontar para uma NF-e válida.
- `efd_registro.id_nfe_referenciada` materializa o cruzamento EFD x NF-e, central para a malha fiscal.
- Como explicado na seção 4.2, usaremos **NO ACTION** nas FKs que apontam para `nfe` a partir de `cte` e `efd_registro`, para evitar o erro de múltiplos caminhos de cascata do SQL Server.

7. Roteiro Prático 1 — Construindo o Esquema com Constraints

Objetivo: criar o esquema completo do modelo de referência no SSMS, com todas as constraints comentadas.

Passo a passo:

1. No SSMS, abra uma nova *Query* sobre o banco `curso_integridade_fiscal` (ver seção 5.3).
2. Execute o script abaixo por partes, tabela a tabela (selecione cada bloco e pressione F5), observando o painel **Messages**.
3. Após cada **CREATE TABLE**, use o **Object Explorer** (clique direito na tabela → **Refresh**) para visualizar a tabela recém-criada e conferir suas *Keys* e *Constraints*.
4. Para cada constraint, escreva num caderno de anotações **qual tipo de integridade ela protege** (domínio, entidade, referencial ou regra de negócio).

```
USE curso_integridade_fiscal;  
GO
```

```
-- 1. CONTRIBUINTE
CREATE TABLE contribuinte (
    id_contribuinte    BIGINT IDENTITY(1,1) NOT NULL,
    cnpj_cpf           VARCHAR(14) NOT NULL,
    razao_social       VARCHAR(150) NOT NULL,
    uf                 CHAR(2) NOT NULL,
    situacao_cadastral VARCHAR(20) NOT NULL CONSTRAINT df_contribuinte_situacao
DEFAULT ('ATIVO'),
    CONSTRAINT pk_contribuinte PRIMARY KEY (id_contribuinte),
    CONSTRAINT uq_contribuinte_doc UNIQUE (cnpj_cpf),
    CONSTRAINT ck_cnpj_cpf_tamanho CHECK (LEN(cnpj_cpf) IN (11,14)),
    CONSTRAINT ck_situacao CHECK (situacao_cadastral IN
('ATIVO','SUSPENSO','BAIXADO','INAPTO'))
);
GO
```

```
-- 2. NF-e
```

```
CREATE TABLE nfe (
    id_nfe              BIGINT IDENTITY(1,1) NOT NULL,
    chave_aceso         CHAR(44) NOT NULL,
    id_contribuinte_emitente BIGINT NOT NULL,
    id_contribuinte_destinatario BIGINT NOT NULL,
    data_emissao        DATE NOT NULL,
    valor_total         NUMERIC(15,2) NOT NULL,
    situacao            VARCHAR(20) NOT NULL CONSTRAINT
df_nfe_situacao DEFAULT ('AUTORIZADA'),
    CONSTRAINT pk_nfe PRIMARY KEY (id_nfe),
    CONSTRAINT uq_nfe_chave UNIQUE (chave_aceso),
    CONSTRAINT ck_nfe_chave_formato CHECK (
        LEN(chave_aceso) = 44 AND chave_aceso NOT LIKE '%[^0-9]%'
    ),
    CONSTRAINT ck_nfe_valor_positivo CHECK (valor_total >= 0),
    CONSTRAINT ck_nfe_data CHECK (data_emissao <= CAST(GETDATE() AS DATE)),
    CONSTRAINT ck_nfe_situacao CHECK (situacao IN
('AUTORIZADA','CANCELADA','DENEGADA','INUTILIZADA')),
    CONSTRAINT fk_nfe_emitente FOREIGN KEY (id_contribuinte_emitente)
        REFERENCES contribuinte (id_contribuinte),
    CONSTRAINT fk_nfe_destinatario FOREIGN KEY (id_contribuinte_destinatario)
        REFERENCES contribuinte (id_contribuinte)
);
GO
```

```
-- 3. ITEM DA NF-e
```

```
CREATE TABLE item_nfe (
    id_item            BIGINT IDENTITY(1,1) NOT NULL,
    id_nfe             BIGINT NOT NULL,
    ncm                CHAR(8) NOT NULL,
    cfop               CHAR(4) NOT NULL,
    valor_item         NUMERIC(15,2) NOT NULL,
    CONSTRAINT pk_item_nfe PRIMARY KEY (id_item),
    CONSTRAINT ck_item_valor_positivo CHECK (valor_item >= 0),
    CONSTRAINT fk_item_nfe FOREIGN KEY (id_nfe)
        REFERENCES nfe (id_nfe) ON DELETE CASCADE
);
GO
```

```
-- 4. NFC-e (estrutura semelhante à NF-e, sem destinatário obrigatório)
```

```
CREATE TABLE nfce (
    id_nfce            BIGINT IDENTITY(1,1) NOT NULL,
```

```

chave_acesso          CHAR(44)          NOT NULL,
id_contribuinte_emitente BIGINT          NOT NULL,
data_emissao          DATE              NOT NULL,
valor_total           NUMERIC(15,2)     NOT NULL,
situacao              VARCHAR(20)       NOT NULL CONSTRAINT df_nfce_situacao
DEFAULT ('AUTORIZADA'),
CONSTRAINT pk_nfce PRIMARY KEY (id_nfce),
CONSTRAINT uq_nfce_chave UNIQUE (chave_acesso),
CONSTRAINT ck_nfce_chave_formato CHECK (
    LEN(chave_acesso) = 44 AND chave_acesso NOT LIKE '%[^0-9]%'
),
CONSTRAINT ck_nfce_valor_positivo CHECK (valor_total >= 0),
CONSTRAINT ck_nfce_situacao CHECK (situacao IN
('AUTORIZADA', 'CANCELADA', 'DENEGADA')),
CONSTRAINT fk_nfce_emitente FOREIGN KEY (id_contribuinte_emitente)
    REFERENCES contribuinte (id_contribuinte)
);
GO

-- 5. ITEM DA NFC-e
CREATE TABLE item_nfce (
    id_item            BIGINT IDENTITY(1,1) NOT NULL,
    id_nfce            BIGINT              NOT NULL,
    ncm                CHAR(8)            NOT NULL,
    cfop               CHAR(4)            NOT NULL,
    valor_item         NUMERIC(15,2)     NOT NULL,
    CONSTRAINT pk_item_nfce PRIMARY KEY (id_item),
    CONSTRAINT ck_item_nfce_valor_positivo CHECK (valor_item >= 0),
    CONSTRAINT fk_item_nfce FOREIGN KEY (id_nfce)
        REFERENCES nfce (id_nfce) ON DELETE CASCADE
);
GO

-- 6. CT-e
CREATE TABLE cte (
    id_cte            BIGINT IDENTITY(1,1) NOT NULL,
    chave_acesso      CHAR(44)          NOT NULL,
    id_contribuinte_emitente BIGINT      NOT NULL,
    id_nfe_vinculada  BIGINT            NULL,    -- pode ser NULL: nem todo
CT-e vincula uma única NF-e
    valor_frete       NUMERIC(15,2)     NOT NULL,
    situacao          VARCHAR(20)       NOT NULL CONSTRAINT
df_cte_situacao DEFAULT ('AUTORIZADO'),
    CONSTRAINT pk_cte PRIMARY KEY (id_cte),
    CONSTRAINT uq_cte_chave UNIQUE (chave_acesso),
    CONSTRAINT ck_cte_frete_positivo CHECK (valor_frete >= 0),
    CONSTRAINT ck_cte_situacao CHECK (situacao IN
('AUTORIZADO', 'CANCELADO', 'DENEGADO')),
    CONSTRAINT fk_cte_emitente FOREIGN KEY (id_contribuinte_emitente)
        REFERENCES contribuinte (id_contribuinte),
    -- NO ACTION: nfe já recebe cascade/participação de outras FKs (item_nfe) –
    evita erro de múltiplos caminhos
    CONSTRAINT fk_cte_nfe FOREIGN KEY (id_nfe_vinculada)
        REFERENCES nfe (id_nfe) ON DELETE NO ACTION
);
GO

-- 7. REGISTRO DE EFD
CREATE TABLE efd_registro (

```

```

id_registro          BIGINT IDENTITY(1,1) NOT NULL,
id_contribuinte      BIGINT          NOT NULL,
tipo_registro        VARCHAR(10)     NOT NULL,    -- ex.: 'C100', 'C170'
periodo_apuracao     CHAR(7)         NOT NULL,    -- formato 'MM/AAAA'
id_nfe_referenciada  BIGINT          NULL,
CONSTRAINT pk_efd_registro PRIMARY KEY (id_registro),
CONSTRAINT ck_perodo_formato CHECK (
    SUBSTRING(periodo_apuracao,3,1) = '/'
    AND LEN(periodo_apuracao) = 7
    AND CAST(LEFT(periodo_apuracao,2) AS INT) BETWEEN 1 AND 12
),
CONSTRAINT fk_efd_contribuinte FOREIGN KEY (id_contribuinte)
    REFERENCES contribuinte (id_contribuinte),
CONSTRAINT fk_efd_nfe FOREIGN KEY (id_nfe_referenciada)
    REFERENCES nfe (id_nfe) ON DELETE NO ACTION
);
GO

```

Checkpoint de discussão em turma: por que `item_nfe` e `item_nfce` usam `ON DELETE CASCADE`, mas as demais FKs usam `NO ACTION`? (Resposta esperada: um item não existe fora do documento que o contém — é parte do "todo"; já um contribuinte, ou uma NF-e vinculada a CT-e/EFD, não deve ser excluído silenciosamente, sob risco de perda de histórico fiscal.)

8. Roteiro Prático 2 — Testando Violações de Integridade

Objetivo: provocar deliberadamente erros de constraint no SSMS e reconhecer, no painel **Messages**, o código de erro e a constraint violada.

1. Insira dois contribuintes válidos.
2. Execute os comandos abaixo, um de cada vez, e registre a mensagem retornada pelo SSMS:

```

-- (a) Violação de integridade de domínio
INSERT INTO contribuinte (cnpj_cpf, razao_social, uf)
VALUES ('12345', 'Empresa Teste LTDA', 'PB');
-- Esperado: Msg 547 – CHECK ck_cnpj_cpf_tamanho

-- (b) Violação de integridade de entidade
INSERT INTO contribuinte (cnpj_cpf, razao_social, uf)
VALUES ('12345678000199', 'Empresa Teste LTDA', 'PB');

INSERT INTO contribuinte (cnpj_cpf, razao_social, uf)
VALUES ('12345678000199', 'Empresa Duplicada LTDA', 'PB');

```

```

-- Esperado: Msg 2627 – violação de UNIQUE (uq_contribuinte_doc)

-- (c) Violação de integridade referencial
INSERT INTO nfe (chave_acesso, id_contribuinte_emitente,
id_contribuinte_destinatario,
                data_emissao, valor_total)
VALUES ('25260712345678000199550010000000011000000015',
        9999, 9999, CAST(GETDATE() AS DATE), 100.00);
-- Esperado: Msg 547 – FOREIGN KEY fk_nfe_emitente (id_contribuinte_emitente 9999
não existe)

-- (d) Violação de regra de negócio (CHECK)
INSERT INTO nfe (chave_acesso, id_contribuinte_emitente,
id_contribuinte_destinatario,
                data_emissao, valor_total)
VALUES ('25260712345678000199550010000000021000000016',
        1, 2, CAST(GETDATE() AS DATE), -500.00);
-- Esperado: Msg 547 – CHECK ck_nfe_valor_positivo

-- (e) Tentativa de excluir um contribuinte que já emitiu documentos
DELETE FROM contribuinte WHERE id_contribuinte = 1;
-- Esperado: Msg 547 – FOREIGN KEY (NO ACTION impede a exclusão)

```

3. Em grupo, preencha a tabela abaixo:

Comando	Constraint violada	Tipo de integridade	Código de erro (SSMS)
(a)			
(b)			
(c)			
(d)			
(e)			

9. Roteiro Prático 3 — Regras de Negócio Fiscais com CHECK e Triggers

Algumas regras não cabem em um **CHECK** de coluna simples porque envolvem **múltiplas linhas ou múltiplas tabelas**. Nesses casos, usamos **TRIGGER**.

Em T-SQL, uma trigger **AFTER INSERT**, **UPDATE**, **DELETE** tem acesso às tabelas virtuais **inserted** e **deleted**, contendo as linhas afetadas pela operação.

Regra de negócio: "A soma dos valores dos itens de uma NF-e deve ser igual ao valor total do documento."

```
USE curso_integridade_fiscal;
GO

CREATE TRIGGER trg_valida_total_nfe
ON item_nfe
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @id_nfe BIGINT;

    -- captura o id_nfe afetado, considerando INSERT, UPDATE ou DELETE
    SELECT @id_nfe = COALESCE(
        (SELECT TOP 1 id_nfe FROM inserted),
        (SELECT TOP 1 id_nfe FROM deleted)
    );

    IF @id_nfe IS NULL
        RETURN;

    DECLARE @soma_itens NUMERIC(15,2);
    DECLARE @total_doc NUMERIC(15,2);

    SELECT @soma_itens = COALESCE(SUM(valor_item), 0)
    FROM item_nfe
    WHERE id_nfe = @id_nfe;

    SELECT @total_doc = valor_total
    FROM nfe
    WHERE id_nfe = @id_nfe;

    IF @soma_itens <> @total_doc
    BEGIN
        RAISERROR('Soma dos itens (%s) difere do valor total da NF-e (%s). Operação
cancelada.',
                16, 1, @soma_itens, @total_doc);
        ROLLBACK TRANSACTION;
    END
END;
GO
```

Regra de negócio: "Um documento cancelado não pode voltar a ser AUTORIZADO" (transição de estado irreversível).

```
CREATE TRIGGER trg_impede_reabertura_nfe
ON nfe
AFTER UPDATE
AS
BEGIN
```



```
SET NOCOUNT ON;
```

```
IF EXISTS (  
    SELECT 1  
    FROM deleted d  
    JOIN inserted i ON i.id_nfe = d.id_nfe  
    WHERE d.situacao = 'CANCELADA' AND i.situacao <> 'CANCELADA'  
)  
BEGIN  
    RAISERROR('NF-e cancelada não pode mudar de situação.', 16, 1);  
    ROLLBACK TRANSACTION;  
END  
END;  
GO
```

Discussão em turma: por que essas duas regras não poderiam ser implementadas apenas com **CHECK**? (Resposta esperada: **CHECK** em SQL Server avalia apenas a própria linha da tabela em que está definido, não agrega valores de outras linhas nem compara o valor antigo com o novo — para isso é necessário **TRIGGER**, que tem acesso às tabelas **inserted/deleted**.)

Nota sobre SET NOCOUNT ON: boa prática em toda trigger T-SQL — evita que a contagem de linhas afetadas interfira em aplicações cliente que dependem do retorno de **@@ROWCOUNT**.

10. Exercícios Práticos

Exercício 1 — Identificando o tipo de integridade (teórico)

Classifique cada situação abaixo como integridade de **domínio**, **entidade**, **referencial** ou **regra de negócio**:

a) Duas NF-e com a mesma chave de acesso de 44 dígitos. b) Um CT-e referenciando uma NF-e com **id_nfe = 500**, mas essa NF-e não existe na base. c) Um campo **cfop** recebendo o valor **'ABCD'** em vez de um código numérico de 4 dígitos. d) Uma NFC-e emitida com data de emissão posterior à data atual. e) Uma NF-e sendo cancelada duas vezes.

Exercício 2 — Modelando o Cadastro de Transportadoras

No SSMS, crie a tabela `transportadora` com:

- `id_transportadora` (PK, `IDENTITY(1,1)`).
- `cnpj` (14 caracteres, único, obrigatório).
- `rntrc` (Registro Nacional de Transportadores Rodoviários de Cargas — 8 dígitos, obrigatório).
- Adicione uma coluna `id_transportadora` em `cte`, com FK para `transportadora`, permitindo `NULL` e `ON DELETE SET NULL`.

(Dica: use `ALTER TABLE cte ADD ...` para adicionar a coluna e a constraint em uma tabela já existente.)

Exercício 3 — Regra de CFOP compatível com o tipo de operação

Adicione uma constraint em `item_nfe` que impeça CFOPs iniciados em '5' (operações dentro do estado) quando o emitente e o destinatário da nota estiverem em UFs diferentes. (Dica: esta regra depende de dados de outra tabela — um `CHECK` de coluna simples é suficiente em T-SQL? Justifique e implemente a solução adequada — `TRIGGER` ou `CHECK` com função escalar.)

Exercício 4 — Cruzamento EFD x NF-e

Escreva uma consulta T-SQL que liste todas as NF-e **autorizadas** que **não possuem** nenhum registro correspondente em `efd_registro` — um indício de possível omissão de escrituração. (Dica: `LEFT JOIN` com `WHERE ... IS NULL`, ou `NOT EXISTS`.)

Exercício 5 — Simulando exclusão em cascata

1. No SSMS, insira uma NF-e com três itens.
2. Exclua a NF-e (`DELETE FROM nfe WHERE ...`).
3. Verifique (com `SELECT`) o que aconteceu com os itens em `item_nfe`.

4. Explique, em termos de integridade referencial, por que isso ocorreu (relacione com a cláusula **ON DELETE CASCADE** usada na criação da tabela).

Exercício 6 — Projetando uma nova regra de negócio

A SEFAZ-PB decidiu que **NFC-e não pode ter valor total acima de R\$ 5.000,00** (regra fiscal hipotética para fins didáticos). Implemente essa regra: a) Primeiro, como **CHECK** de coluna. b) Depois, discuta: e se o limite dependesse do regime tributário do contribuinte emitente (armazenado em **contribuinte**)? O **CHECK** de coluna ainda seria suficiente? Reescreva a solução usando **TRIGGER**, seguindo o padrão do Roteiro Prático 3.

Exercício 7 (Desafio) — Auditoria de integridade em base já populada

Você recebeu uma base de dados legada (sem constraints aplicadas) com tabelas equivalentes a **nfe** e **item_nfe**. Escreva consultas T-SQL para **detectar**, antes de tentar aplicar as constraints: a) Chaves de acesso duplicadas em **nfe** (**GROUP BY ... HAVING COUNT(*) > 1**). b) Itens (**item_nfe**) órfãos, ou seja, sem **nfe** correspondente (**NOT EXISTS**). c) NF-e com **valor_total** negativo.

*(Dica: esse é um passo obrigatório antes de adicionar **PRIMARY KEY**, **FOREIGN KEY** ou **CHECK** em uma tabela já populada — o SQL Server rejeitará **ALTER TABLE ... ADD CONSTRAINT** se houver dados que a violem, a menos que se use **WITH NOCHECK**, o que **não é recomendado** em bases fiscais por deixar dados inconsistentes sem validação.)*

Exercício 8 — Explorando o SSMS

1. Abra o **Object Explorer** e localize a tabela **nfe** criada no Roteiro Prático 1.
2. Expanda **Keys** e **Constraints** e liste, em uma tabela no seu caderno, todas as constraints encontradas e seu tipo.
3. Gere o script de uma das **FOREIGN KEY** usando **Script Key as → CREATE To → New Query Editor Window** e cole o resultado como resposta do exercício.
4. Crie um **Database Diagram** (seção 5.6) incluindo as tabelas **contribuinte**, **nfe** e **item_nfe**, e tire um print da tela mostrando as linhas de relacionamento.

11. Estudos de Caso — Erros Reais de Integridade em Ambientes Fiscais

Caso 1 — "NF-e órfã" na importação de EFD Durante a carga de um arquivo EFD, um registro C100 referenciava uma chave de acesso de NF-e que nunca havia sido recebida pela SEFAZ. Sem uma **FOREIGN KEY** (ou validação equivalente antes da carga), o registro foi inserido normalmente, gerando inconsistência na malha fiscal. **Lição:** integridade referencial deve ser validada tanto no banco quanto na rotina de importação (ETL/SSIS).

Caso 2 — Duplicidade de chave de acesso Um reproprocessamento de lote de NF-e, sem constraint **UNIQUE** em **chave_acesso**, resultou em documentos duplicados na base, inflando indevidamente o valor total de operações de um contribuinte em relatórios gerenciais. **Lição:** **UNIQUE** é uma proteção barata comparada ao custo de corrigir relatórios e reproprocessar auditorias.

Caso 3 — Cancelamento fora de sequência Uma aplicação permitiu que um CT-e fosse marcado como **CANCELADO** e, em seguida, revertido para **AUTORIZADO** por um erro de usuário, sem trilha de auditoria. **Lição:** regras de transição de estado (via **TRIGGER**) e auditoria de alterações (tabelas de log, ou **Temporal Tables** do SQL Server) são complementares às constraints estruturais.

Atividade em grupo: discuta com a turma um caso real (anonimizado) já observado no ambiente de trabalho de vocês na SEFAZ-PB e identifique qual constraint teria evitado o problema.

12. Glossário

Termo	Definição
Constraint	Regra declarada no esquema do banco que restringe os valores/relacionamentos permitidos
T-SQL	Transact-SQL, dialeto de SQL utilizado pelo Microsoft SQL Server
SSMS	SQL Server Management Studio, ferramenta gráfica de administração do SQL Server

Termo	Definição
Chave de acesso	Código numérico de 44 dígitos que identifica unicamente um documento fiscal eletrônico
CFOP	Código Fiscal de Operações e Prestações
NCM	Nomenclatura Comum do Mercosul
EFD	Escrituração Fiscal Digital
Malha fiscal	Processo de cruzamento de informações fiscais para identificar inconsistências
Cascade	Ação que propaga uma exclusão/alteração da tabela "pai" para a tabela "filha"
Trigger	Rotina executada automaticamente pelo SQL Server em resposta a eventos de INSERT/UPDATE/DELETE
inserted / deleted	Tabelas virtuais disponíveis dentro de uma trigger T-SQL, contendo as linhas afetadas

13. Referências

- Documentação oficial da Microsoft — "CREATE TABLE (Transact-SQL)": <https://learn.microsoft.com/sql/t-sql/statements/create-table-transact-sql>
- Documentação oficial da Microsoft — "CREATE TRIGGER (Transact-SQL)": <https://learn.microsoft.com/sql/t-sql/statements/create-trigger-transact-sql>
- Documentação oficial da Microsoft — "Uniqueness Constraints and Check Constraints": <https://learn.microsoft.com/sql/relational-databases/tables/unique-constraints-and-check-constraints>
- Documentação do SQL Server Management Studio: <https://learn.microsoft.com/sql/ssms/sql-server-management-studio-ssms>
- Manual de Orientação do Contribuinte — NF-e (Encontros Nacionais de Administradores Tributários — ENCAT): <https://www.nfe.fazenda.gov.br>
- Manual de Orientação do Contribuinte — CT-e: <https://www.cte.fazenda.gov.br>
- Guia Prático da EFD ICMS/IPI: <http://sped.rfb.gov.br>
- ELMASRI, R.; NAVATHE, S. *Sistemas de Banco de Dados*. 7ª ed. Pearson.
- DATE, C. J. *Introdução a Sistemas de Banco de Dados*. 8ª ed. Campus.

